

From Ackermann to ε_0 : a Tutorial Walk-Through of `contender_6`

(name-the-biggest-number, contender 6)

Abstract

This article is a self-contained tutorial on the construction of `contender_6`, a new entry to the Coq-formalized *name-the-biggest-number* contest of github.com/codyroux/name-the-biggest-number. The previous contender, `contender_5`, is the largest natural number that any closed term of System T (simply-typed λ -calculus with `NatRec`) of abstract-syntax-tree depth at most 42 evaluates to. We extend the syntax with a single new primitive, `tGrow`, whose semantics is the function f_{ε_0} from the fast-growing hierarchy, and define `contender_6` as the analogous depth-bounded maximum at depth 44 in the extended language. The proof of strict inequality `contender_5 < contender_6` proceeds by embedding the previous contender's hidden maximizer term into the new language, using `tGrow` as one extra application layer to break through System T's ε_0 ceiling.

The tutorial presents in turn: (1) the contest setting and the *don't-be-lazy* rule; (2) Brouwer ordinal notations and why they turn ordinal recursion into ordinary structural recursion; (3) the fast-growing hierarchy and its closure properties; (4) the new object language $\mathcal{L}_{\text{Grow}}$ and its denotational semantics; (5) a Tait-style logical relation that proves the embedding preserves evaluation *without* function extensionality; and (6) the final witness term and the chain of inequalities. Every step is accompanied by the corresponding Rocq/Coq formalization, taken verbatim from the inlined `Contender.v`.

Contents

1 The contest in a nutshell

1.1 Big numbers, formally

In Scott Aaronson’s 1999 essay *Who Can Name the Bigger Number?* [?] the rules of the eponymous game are a single line on a blackboard: each contestant writes a natural number, and the larger one wins. The interesting twist is that “larger” is not measured in digits or magnitude of the numeral, but in the *description* of the number. Tower notation beats exponential, primitive recursion beats tower, the Ackermann function beats primitive recursion, and so on, climbing through the ordinals of proof theory.

The Coq repository `codyroux/name-the-biggest-number` formalizes this game in a small but precise way:

- Numbers must be *constructive*: no busy-beaver shenanigans, no oracles, no inconsistent axioms. Each contender is a closed term of type `nat`.
- Each contender comes with a *formal Coq proof* that it strictly exceeds the previous one.
- Type-checking the definition takes at most 15 seconds and the proof at most 60 seconds on a reasonable machine.
- No axioms.
- The crucial *don’t-be-lazy* rule: the new contender `contender_N` must *not* be defined by literally invoking the previous `contender_N-1` as a numeral. The spirit of the rule is that a one-line victory of the form “`contender_5 + 1`” should be ineligible. As we will see, this is more subtle than it sounds.

The current chain of contenders is:

#	Definition	Order of magnitude
1	42	a two-digit literal
2	a tower of $42^{\dots^42}$	several thousand 42s tall
3	<code>tower_of_power 10000</code>	a 42-tower of height 10001
4	<code>ack 5 42 10002</code>	an Ackermannian giant
5	largest closed System T term, depth ≤ 42	dwarfs every f_α for $\alpha < \varepsilon_0$
6	the present submission	$\geq f_{\varepsilon_0}(\text{contender_5} + 1)$

2 The chain of previous contenders

We now walk through `contender_1`–`contender_5` in detail, showing the actual definition of each one and the structure of the proof of strict inequality `contender_N-1 < contender_N`. The five steps span six order-of-magnitude regimes – literals, towers of exponents, primitive recursion on towers, the Ackermann hierarchy, and finally a depth-bounded enumeration of closed System T terms. A reader already familiar with the upstream chain can safely skip to §??.

The first four contenders are kept in the file `Graveyard.v` of the upstream repository; `contender_4` and `contender_5` together with the proof `contender_4 < contender_5` live in `Contender.v`. Both files are axiom-free; we quote each definition and the proof structure verbatim.

2.1 `contender_1 = 42`

The chain bottoms out at a literal, after a degenerate `contender_0 = 0`:

```
Definition contender_0 : nat := 0.
Definition contender_1 : nat := 42.
```

```
Theorem contender_0_lt_contender_1 : contender_0 < contender_1.
Proof. unfold contender_0, contender_1. auto with arith. Qed.
```

The proof is a one-liner: `auto with arith` closes $0 < 42$ on its own. This contender is the bottom of the well; it exists mostly as a pedagogical anchor.

2.2 contender_2: a tower of 42's

`contender_2` is a *literal* numeric expression: a right-associated chain of $42^{42^{\dots^{42}}}$ of several thousand levels, parsed by Coq into one giant `nat` term:

```
Time Definition contender_2 : nat :=
  42^42^42^42^...^42. (* thousands of 42s *)
```

The proof of strict inequality is a tactic that repeatedly applies the monotonicity of a^b in b :

```
Theorem contender_1_lt_contender_2 : contender_1 < contender_2.
Proof.
  Time cbv delta [contender_1 contender_2];
  repeat match goal with
    | _ => progress eapply Nat.pow_lt_mono_r; [solve [lia]]|
    | ⊢ ?x < ?y =>
      match y with
      | context [x ^ x] =>
        progress erewrite ← Nat.pow_1_r at 1
      end
    | _ => progress eapply Nat.pow_gt_1; [solve [lia]]|
    | _ => progress eapply Nat.pow_nonzero; solve [lia]
  end.
Time Qed.
```

The author's own comment records that the `Definition` took ~ 2 seconds and the proof script roughly 50 seconds – well within today's 15s/60s budget but not by much. The `Time` markers are a reminder that the upstream rules *are* measured, not aspirational.

2.3 contender_3: tower_of_power 10000

The third contender introduces the first explicit recursion in the chain. Define a tetration-like family of natural numbers:

```
Definition tower_of_power (n : nat) : nat :=
  Nat.recursion 42 (fun _ k => 42 ^ k) n.

Definition contender_3 := tower_of_power 10000.
```

Here `Nat.recursion` is the standard library's primitive recursor `nat_rect` specialised to a non-dependent codomain. Unrolling: `tower_of_power 0 = 42`, `tower_of_power (n+1) = 42tower_of_power n`, so `contender_3` is a tower of 42 ten thousand and one levels tall – vastly more than the few-thousand-tall `contender_2`.

The proof relies on a custom Ltac `reify_tower` that matches the iterated $42^{}$ pattern in `contender_2` and rewrites it as `tower_of_power k` for the appropriate k , after which a monotonicity lemma about `tower_of_power` closes the goal:

```
Lemma fund_lemma : forall n m,
  n < m → tower_of_power n < tower_of_power m.
```

```

Theorem contender_2_lt_contender_3 : contender_2 < contender_3.
Proof.
  unfold contender_2.
  Time reify_goal.
  unfold contender_3.
  apply fund_lemma.
  compute; lia.
Qed.

```

The `reify_goal` step is the interesting one: it turns the explicit literal-tower `contender_2` into the symbolic form `tower_of_power k`, which `fund_lemma` can compare to `tower_of_power 10000`.

2.4 contender_4 = ack 5 42 10002

The fourth contender takes the Ackermann jump. The Ackermann function in this repository is defined slightly non-standardly to make the proof of `contender_3 < contender_4` tractable:

```

Fixpoint iter (f : nat → nat) (b : nat) :=
  match b with
  | 0 ⇒ 1
  | S b' ⇒ f (iter f b')
  end.

Fixpoint ack (n a b : nat) :=
  match n with
  | 0 ⇒ S b
  | 1 ⇒ a + b
  | S n' ⇒ iter (ack n' a) b
  end.

Definition contender_4 := ack 5 42 10002.

```

So `ack0` = successor, `ack1` = addition, `ack2` = multiplication+1, `ack3` $\approx$ exponentiation, `ack4` $\approx$ tetration, and `ack5` $\approx$ pentation – one full hierarchy step above what `tower_of_power` can express.

The proof of `contender_3 < contender_4` chains a sequence of bounds:

```

Lemma ack_3 : forall a b, a ^ b ≤ ack 3 a b.

Lemma ack_4 : forall b, tower_of_power b ≤ ack 4 42 (S b).

Theorem contender_3_lt_contender_4 :
  contender_3 < contender_4.

```

`ack_3` bounds exponentiation by `ack3` (i.e. proves `ack3` dominates a^b as a function of b); `ack_4` promotes this to bound the entire `tower_of_power` tower by `ack 4 42`; the final theorem combines `ack_4` with the structural monotonicity of `ack` in its third argument (`mon_ack_in_b`) to bridge from `ack 4 42 k` to `ack 5 42 (k+2)`. The actual proof script is about twenty lines of moderate complexity.

2.5 contender_5: the largest System T term of depth at most 42

The fifth contender is the qualitative leap that motivates this whole article. Instead of naming a specific large number, it names *the largest number that any closed System T term of depth at most 42 evaluates to*.

2.5.1 The System T object language

The repository defines a small dependently-evaluated object language: simply-typed lambda calculus with a base type `tpNat` and a `NatRec` primitive. The types are

```
Inductive type :=
| tpNat : type
| tpArr : type → type → type.
```

interpreted as Coq types via a small Fixpoint:

```
Fixpoint interp_type (tp : type) : Type :=
  match tp with
  | tpNat      ⇒ nat
  | tpArr t1 t2 ⇒ interp_type t1 → interp_type t2
  end.
```

Terms are de Bruijn-indexed, with the slightly unusual convention that index 0 is the *outermost* binder (a reversed-de Bruijn or de Bruijn-*level* convention):

```
Inductive term :=
| tVar    (x : nat)
| tLam    (A B : type) (body : term)
| tApp    (t1 t2 : term)
| t0
| tS
| tNatRec (R : type).
```

The interpreter `interp_term` maps a term and an environment of values to a dependent pair of a type tag and a value:

```
Definition interp_term :
  forall (e : list {tp : type & interp_type tp})
    (t : term),
    {tp : type & interp_type tp}.
```

For `tApp`, the type tag of the function is destructured; if it is not `tpArr`, a default `error` value is returned. For `tNatRec`, the term reifies Coq’s `Nat.recursion` at the requested return type. The full body uses `refine` plus a small `cast_impl` coercion that reconciles “the function expected an argument of type A , but its actual argument has type A' ” situations – if the type tags do not match, the coercion falls through to `error`.

The top-level evaluator extracts a `nat` from a closed `tpNat`-typed result and returns 0 otherwise:

```
Definition eval (t : term) : nat :=
  let (tp, res) := interp_term nil t in
  match tp as t0 return interp_type t0 → nat with
  | tpNat      ⇒ fun n ⇒ n
  | tpArr _ _  ⇒ fun _ ⇒ 0
  end res.
```

2.5.2 Term depth and the depth-bounded enumerator

Term depth is the abstract-syntax-tree depth, with a special treatment of variable indices and types that lets the finite-enumeration argument go through:

```

Definition nat_depth (n : nat) := S n.

Fixpoint type_depth (t : type) : nat :=
  match t with
  | tpNat      ⇒ 1
  | tpArr t1 t2 ⇒ S (max (type_depth t1) (type_depth t2))
  end.

Fixpoint term_depth (t : term) : nat :=
  match t with
  | tVar x      ⇒ S (nat_depth x)
  | tLam A B body ⇒ S (max (max (type_depth A)
                                (type_depth B))
                        (term_depth body))
  | tApp t1 t2   ⇒ S (max (term_depth t1) (term_depth t2))
  | t0           ⇒ 1
  | tS           ⇒ 1
  | tNatRec R     ⇒ S (type_depth R)
  end.

```

Crucially, `nat_depth n` is *not* bounded by any constant; it is just $n + 1$. So for any depth bound d , only finitely many natural numbers n have `nat_depth n` $\leq d$ – namely $\{0, 1, \dots, d - 1\}$. Similarly only finitely many types have `type_depth t` $\leq d$, and only finitely many closed terms have `term_depth t` $\leq d$.

The repository realizes “finitely many” as concrete enumeration functions:

```

Fixpoint natsUpTo (n : nat) : list nat :=
  match n with
  | 0    ⇒ []
  | S m  ⇒ m :: natsUpTo m
  end.

Fixpoint typesUpTo (n : nat) : list type :=
  match n with
  | 0    ⇒ []
  | S m  ⇒
    let r := typesUpTo m in
    tpNat :: List.map (fun '(t1, t2) ⇒ tpArr t1 t2)
                  (list_prod r r)
  end.

Fixpoint termsUpTo (n : nat) : list term :=
  match n with
  | 0    ⇒ []
  | S m  ⇒
    List.map tVar (natsUpTo m) ++
    List.map (fun '(A, B, body) ⇒ tLam A B body)
              (list_prod (list_prod (typesUpTo m) (typesUpTo m))
                        (termsUpTo m)) ++
    List.map (fun '(t1, t2) ⇒ tApp t1 t2)
              (list_prod (termsUpTo m) (termsUpTo m)) ++
    [t0] ++ [tS] ++
    List.map tNatRec (typesUpTo m)
  end.

```

The three correctness lemmas characterize each enumerator as “everything of depth at most n

appears in the list, and nothing else does”:

```

Lemma natsUpTo_correct : forall n m, nat_depth m ≤ n ↔ In m (natsUpTo n).
Lemma typesUpTo_correct : forall n t, type_depth t ≤ n ↔ In t (typesUpTo n).
Lemma termsUpTo_correct : forall n t, term_depth t ≤ n ↔ In t (termsUpTo n).

```

The proof of `termsUpTo_correct` is a routine induction with seven cases (one per term constructor); each case navigates to the right `++` segment and proves membership via repeated `in_map` / `in_prod` reductions plus an appeal to `natsUpTo_correct` or `typesUpTo_correct` or the IH.

2.5.3 `maxBy`: argmax over a list

The maximum-eval combinator is polymorphic over the element type:

```

Fixpoint maxBy {T : Type}
  (f : T → nat) (currentMax : nat)
  (currentBest : T) (l : list T) : T :=
  match l with
  | nil      ⇒ currentBest
  | cons h t ⇒
    if currentMax <? f h
    then maxBy f (f h) h t
    else maxBy f currentMax currentBest t
  end.

```

At each step `maxBy` either keeps the current best (if no element of the list exceeds it) or upgrades to a new best. The three key properties are recorded as Coq lemmas:

```

Lemma maxBy_In : forall f l currentMax currentBest,
  currentMax = f currentBest →
  In (maxBy f currentMax currentBest l) l ∨
  maxBy f currentMax currentBest l = currentBest.

Lemma maxBy_at_least_currentMax : forall f l currentMax currentBest,
  f currentBest = currentMax →
  currentMax ≤ f (maxBy f currentMax currentBest l).

Lemma lowerbound_maxBy : forall f x l currentMax currentBest,
  In x l →
  f currentBest = currentMax →
  f x ≤ f (maxBy f currentMax currentBest l).

```

`maxBy_In` is the dichotomy “the result is either an element of the list, or the initial best”; `maxBy_at_least_currentMax` says the final f -value is at least the initial f -value; `lowerbound_maxBy` extends this to: every element of the list is bounded above by the final f -value.

The number we are after is the eval of the depth-42 argmax:

```

Definition largest_of_depth (n : nat) : term :=
  maxBy eval 0 t0 (termsUpTo n).

Definition largest_STLCNatRec_nat_of_depth (n : nat) : nat :=
  eval (largest_of_depth n).

Definition contender_5 : nat :=
  largest_STLCNatRec_nat_of_depth 42.

```

2.5.4 Reification: from ack 5 42 10002 to a System T term

To prove `contender_4 < contender_5`, the critical step is showing that `contender_4` itself is the eval of *some* closed System T term of depth at most 42. We cannot just unfold `ack 5 42 10002` into a numeral and write that as `tS(tS(⋯ t0))` in the object language; the resulting AST would have one node per unit in the value, and `ack 5 42 10002` is astronomically larger than any quantity that could fit in a finite depth budget – or, for that matter, in the observable universe. Instead, the repository reifies the *Coq function* `ack` into a System T term whose `interp_term` is provably equal to it, exploiting the fact that nested `Nat.recursions` let a shallow tree denote a very large number.

The recipe proceeds in three layers. First, define a System T-friendly copy of each arithmetic operation as a Coq function written exclusively in terms of `Nat.recursion`, and prove it equal to the standard library version:

```

Definition iter' (f : nat → nat) : nat → nat :=
  Nat.recursion 1 (fun pred rec ⇒ f rec).
Lemma iter'_equiv : forall f b, iter' f b = iter f b.

Definition add' : nat → nat → nat :=
  Nat.recursion (fun m ⇒ m)
    (fun pred rec m ⇒ S (rec m)).
Lemma add'_equiv : forall n m, add' n m = Nat.add n m.

Definition mul' : nat → nat → nat :=
  Nat.recursion (fun m ⇒ 0)
    (fun pred rec m ⇒ add' m (rec m)).
Lemma mul'_equiv : forall n m, mul' n m = Nat.mul n m.

Definition pow' (n : nat) : nat → nat :=
  Nat.recursion 1 (fun pred rec ⇒ mul' n rec).
Lemma pow'_equiv : forall n m, pow' n m = Nat.pow n m.

Definition ack' (n : nat) : nat → nat → nat := ...
Lemma ack'_equiv : forall n a b, ack' n a b = ack n a b.

```

The key feature of these primed definitions is that they are *written* as nested `Nat.recursions` and ordinary λ -abstractions – precisely the operations that System T can faithfully express.

Second, an existential witness lemma promises that `ack'` *can* be obtained as the interpretation of some closed System T term, and a custom Ltac `t` (defined elsewhere in the file) repeatedly applies `interp_tApp`, `interp_tLam`, etc. to assemble that term:

```

Lemma ack'_reification_helper : exists res,
  interp_term nil res
  = existT _ (tpArr tpNat (tpArr tpNat (tpArr tpNat tpNat))) ack'.
Proof.
  eexists.
  unfold ack'.
  repeat t.
  all: reflexivity.
Defined.

Definition ack_reified : term.
  let r := eval unfold ack'_reification_helper
    in ack'_reification_helper in
  match r with
  | ex_intro _ ?x _ ⇒ exact x

```



```
end.
Defined.
```

The point of the `Defined.` (rather than `Qed.`) is that the existential witness must be reduce-able by Coq’s kernel: we will need its concrete `term` value when computing `term_depth (ack_reified)`.

Third, the same trick is repeated for `contender_4` itself, yielding a closed System T term `contender_4''_reified` whose `eval` is `contender_4`:

```
Definition contender_4'' : nat. ...

Lemma contender_4''_equiv : contender_4'' = contender_4.

Definition contender_4''_reified : term. ...

Lemma eval_contender_4''_reified :
  eval contender_4''_reified = contender_4''.
```

The depth of `contender_4''_reified` can be computed by `cbv`; running this in the file shows it to be a small constant well below 42. Concretely, `Eval cbv in (term_depth contender_4''_reified)` prints a number around the 30s, and that is what gives the proof its room to breathe.

2.5.5 The proof `contender_4 < contender_5`

With the reification in hand, the actual proof is a clean four-step inequality:

```
Theorem contender_4_lt_contender_5 : contender_4 < contender_5.
Proof.
  rewrite <- contender_4''_equiv.
  rewrite <- eval_contender_4''_reified.
  unfold contender_5, largest_STLCNatRec_nat_of_depth.
  eapply Nat.le_lt_trans.
  - eapply upperbound_eval.
  - eapply largest_of_depth_strictly_monotone.
    cbv. lia.
Qed.
```

Reading the script:

- (1) `rewrite <- contender_4''_equiv` replaces the goal

$$\text{contender_4} < \text{contender_5}$$

by

$$\text{contender_4''} < \text{contender_5}.$$

- (2) `rewrite <- eval_contender_4''_reified` replaces `contender_4''` on the LHS by

$$\text{eval}(\text{contender_4''_reified}).$$

- (3) `unfold` on the RHS exposes `eval(largest_of_depth 42)`.

- (4) Two helper lemmas finish the job. `upperbound_eval` states “every closed term’s `eval` is bounded by the `eval` of the depth-bounded `argmax` at *the term’s own depth*”:

```
Lemma upperbound_eval : forall t,
  eval t ≤ eval (largest_of_depth (term_depth t)).
```

Applying it gives $\text{eval}(\text{contender_4''_reified}) \leq \text{eval}(\text{largest_of_depth}(\text{term_depth} \text{ contender_4''_reified}))$. And `largest_of_depth_strictly_monotone` states that the `argmax` strictly grows with depth (whenever both depths are positive):

```
Lemma largest_of_depth_strictly_monotone : forall n1 n2,
  0 < n1 < n2 →
  eval (largest_of_depth n1) < eval (largest_of_depth n2).
```

The final `cbv. lia` reduces the literal depth of `contender_4''_reified` to a concrete numeral (somewhere in the 30s) and discharges $0 < \text{that} < 42$ by `lia`.

Recapping the four lemmas of the previous chain. For our own proof of `contender_5 < contender_6`, we will need exactly four facts from the inherited machinery:

- `termsUpTo_correct` – every closed term t with $\text{depth}(t) \leq n$ appears in `termsUpTo n`;
- `maxBy_In` – `maxBy` returns either the initial best or some element of the list;
- `lowerbound_maxBy` – every element of the list is bounded above by the maximum;
- `largest_of_depth_monotone` – the maximum is monotone in the depth bound.

These four lemmas plus the `interp_term/eval` infrastructure are everything we will need. The reification, `ack_reified`, and `contender_4''_reified` are spectator citizens: they are invoked only inside the `contender_4 < contender_5` proof, never re-imported by us.

3 Why contender_5+1 is not enough

The naive way to defeat `contender_5` is

$$\text{contender_6} \stackrel{?}{=} \text{contender_5} + 1.$$

This “definition” is morally banned by the don’t-be-lazy rule: the *definition* mentions `contender_5` directly, even though the proof would be one line. More subtly, *any* construction whose definitional unfolding contains the symbol `largest_STLCNatRec_nat_of_depth` (`contender_5`’s search machinery) is borderline-violating the rule, and the maintainer has historically rejected such submissions on those grounds.

Our challenge, then, is to define a number whose unfolded form does *not* mention any of `contender_5`’s machinery, while still proving it larger than `contender_5`. This is exactly the *fresh-engine* pattern.

4 Strategy: a fresh engine for one extra layer

4.1 The bound on System T

System T is famously bounded by the proof-theoretic ordinal of Peano arithmetic, ε_0 . Concretely:

Every closed System T term t of type Nat satisfies $\text{eval}(t) \leq f_\alpha(c)$ for some $\alpha < \varepsilon_0$ and constant c depending only on t .

So if we want a number $N > \text{contender_5}$, the easiest way *not* to be lazy is to use, in the definition, a function that *cannot* be denoted by any closed System T term: the fast-growing-hierarchy ordinal- ε_0 function

$$\text{BigGrow}(n) := f_{\varepsilon_0}(n).$$

We will not actually *prove* this bound on System T inside `Contender.v` – a full proof of the connector lemma “ $\text{eval}(t) \leq f_{\varepsilon_0}(\text{depth}(t))$ ” would be a substantial exercise – but we do not need it. All we need is one explicit *lower bound* witness, and the witness we will use makes the inequality work for purely combinatorial reasons.

4.2 The fresh-engine pattern

The construction is sketched in three lines:

- (1) Define a fresh object language $\mathcal{L}_{\text{Grow}} = \text{System T} + \text{tGrow}$, where `tGrow` is a unary function symbol of type $\text{Nat} \rightarrow \text{Nat}$ interpreted as `BigGrow`.
- (2) Define `contender_6` $:= \text{largest}_{\mathcal{L}_{\text{Grow}}} (44)$, the maximum of $\text{eval}(t)$ over closed $\mathcal{L}_{\text{Grow}}$ terms of depth at most 44.
- (3) For the proof, retrieve the *existential* witness t^* realizing `contender_5` (a closed $\text{depth}(t^*) \leq 42$ System T term with $\text{eval}(t^*) = \text{contender_5}$, guaranteed to exist by `maxBy_In`), embed it into $\mathcal{L}_{\text{Grow}}$ as $\text{embed}(t^*)$, and use the depth-44 witness term `tGrow (tS (embed t*))`.

The key observation is that the *definition* of `contender_6` mentions only the depth-bounded enumerator, `maxBy`, and `eval` of the new language – no `contender_5` machinery whatsoever. The proof is allowed to use the existential maximizer, because the maintainer’s rule prohibits laziness in *definitions*, not in *proofs*.

5 A primer on Brouwer ordinal notations

The next three sections build up the function `BigGrow`. Readers familiar with the fast-growing hierarchy can skim them.

5.1 The data

Brouwer ordinal notations represent ordinals as well-founded trees:

```
Inductive Brouwer : Type :=
| Bz      : Brouwer
| Bsucc : Brouwer → Brouwer
| Blim  : (nat → Brouwer) → Brouwer.
```

The three constructors mean:

- `Bz` is the ordinal 0.
- `Bsucc a` is the successor $a + 1$.
- `Blim f` is the limit $\sup_n f(n)$, with the *fundamental sequence* $(f(0), f(1), \dots)$ baked into the data of the term.

The fundamental-sequence-as-data presentation is the trick that makes ordinal recursion in Coq painless: any `Fixpoint` on `Brouwer` that recurses on a strict subterm is structurally total, and Coq’s W-type guard accepts the limit case by treating $f(n)$, for an arbitrary fresh `n`, as a structural subterm of `Blim f`.

Remark 5.1. This is *not* an extensional presentation of ordinals: two different fundamental sequences for the same supremum give two distinct `Brouwer` terms. But for fast-growing-hierarchy

purposes the choice of fundamental sequence is exactly what we want to commit to: f_{ω^2} and $f_{\omega \cdot \omega}$ are extensionally different on a Brouwer presentation.

5.2 Embedding the natural numbers

The natural numbers embed into Brouwer ordinals as iterated successors:

```
Fixpoint nat_to_B (n : nat) : Brouwer :=
  match n with
  | 0    => Bz
  | S n' => Bsucc (nat_to_B n')
  end.

Definition omega : Brouwer := Blim nat_to_B.
```

So ω is the limit of the natural-number embedding – exactly the ordinal-theoretic definition.

5.3 Ordinal arithmetic

Addition, multiplication, and ω -exponentiation all proceed by structural recursion on the second argument:

```
Fixpoint Badd (a b : Brouwer) : Brouwer :=
  match b with
  | Bz      => a
  | Bsucc b' => Bsucc (Badd a b')
  | Blim f   => Blim (fun n => Badd a (f n))
  end.

Fixpoint Bmul (a b : Brouwer) : Brouwer :=
  match b with
  | Bz      => Bz
  | Bsucc b' => Badd (Bmul a b') a
  | Blim f   => Blim (fun n => Bmul a (f n))
  end.

Fixpoint omega_pow (a : Brouwer) : Brouwer :=
  match a with
  | Bz      => Bsucc Bz
  | Bsucc a' => Bmul (omega_pow a') omega
  | Blim f   => Blim (fun n => omega_pow (f n))
  end.
```

These mirror the textbook recursion equations for Cantor-normal-form ordinals. The reader is invited to convince themselves that `Badd`, `Bmul`, and `omega_pow` pass Coq’s positivity / guard checks: each case recurses only on a structurally smaller argument.

5.4 The ω -tower and ε_0

The ω -tower iterates ω -exponentiation:

```
Fixpoint omega_tower (n : nat) : Brouwer :=
  match n with
  | 0    => Bsucc Bz
  | S n' => omega_pow (omega_tower n')
  end.

Definition epsilon_0 : Brouwer := Blim omega_tower.
```

So

$$\omega \uparrow\uparrow 0 = 1, \quad \omega \uparrow\uparrow (n+1) = \omega^{\omega \uparrow\uparrow n}, \quad \varepsilon_0 = \sup_n (\omega \uparrow\uparrow n).$$

This is the proof-theoretic ordinal of Peano arithmetic, hence the natural ceiling for any system at most as strong as PA. Crucially for us, ε_0 is *above* the ordinal ω^ω , which is itself above any ordinal expressible in System T's normalization.

6 The fast-growing hierarchy and BigGrow

6.1 Definition

The fast-growing hierarchy is a function indexed by ordinals:

$$f_0(n) = n + 1, \quad f_{a+1}(n) = \underbrace{f_a(f_a(\cdots f_a(n)\cdots))}_{n \text{ applications}}, \quad f_\lambda(n) = f_{\lambda[n]}(n),$$

where $\lambda[n]$ is the n -th term of the fundamental sequence at the limit ordinal λ . In Coq, with our Brouwer-data presentation, every $\lambda[n]$ is just $f(n)$ for the function f stored in `Blim f`, so the limit case is one line:

```
Fixpoint FGH (a : Brouwer) (n : nat) : nat :=
  match a with
  | Bz      => S n
  | Bsucc a' => Nat.iter n (FGH a') n
  | Blim f   => FGH (f n) n
  end.
```

Total, structurally recursive on the ordinal argument, no fuel parameter, no `Fix` / `Acc`.

6.2 Sanity checks at small ordinals

For $\alpha < \omega$ we recover textbook arithmetic: $f_0(n) = n + 1$, $f_1(n) = 2n$, $f_2(n) \approx 2^n n$, $f_3(n)$ is roughly tetrahedral, and so on. The first limit ordinal already diagonalizes:

$$f_\omega(n) = f_{\omega[n]}(n) = f_n(n).$$

So $f_\omega(0) = f_0(0) = 1$, $f_\omega(1) = f_1(1) = 2$, $f_\omega(2) = f_2(2) = 8$, $f_\omega(3) = f_3(3)$ is already a number with $\sim 10^8$ bits, and $f_\omega(4)$ is no longer printable in any reasonable amount of memory.

For ε_0 , we have

$$f_{\varepsilon_0}(n) = f_{\varepsilon_0[n]}(n) = f_{\omega \uparrow\uparrow n}(n).$$

A function that, on input n , indexes the fast-growing hierarchy at *the n -fold ω -tower*. This is far beyond what any closed term of System T can compute.

6.3 Wrapping it up

The function `BigGrow` is just f_{ε_0} :

```
Definition BigGrow (n : nat) : nat := FGH epsilon_0 n.
```

Two crude-but-useful monotonicity facts close the section:

```

Lemma Nat_iter_ge :
  forall (f : nat → nat) (k x : nat),
    (forall n, n ≤ f n) →
      x ≤ Nat.iter k f x.

Lemma FGH_ge : forall a n, n ≤ FGH a n.

Lemma BigGrow_ge : forall n, n ≤ BigGrow n.
Lemma BigGrow_gt_S : forall n, n < BigGrow (S n).

```

The last lemma, `BigGrow_gt_S`, is the only growth fact we actually need from the entire Brouwer apparatus. It says ‘*shifting up by one and feeding through `BigGrow` gives a strictly larger value*’, and its proof is two lines: $n < n + 1 \leq \text{BigGrow}(n + 1)$ by `FGH_ge`.

7 The new language $\mathcal{L}_{\text{Grow}}$

7.1 Syntax

$\mathcal{L}_{\text{Grow}}$ is System T plus one fresh constant `tGrow`:

```

Inductive term :=
| tVar (x : nat)
| tLam (A B : type) (body : term)
| tApp (t1 t2 : term)
| t0
| tS
| tNatRec (R : type)
| tGrow.

```

Types are unchanged from `Contender.v`: a base type `tpNat` plus arrow types `tpArr A B`. Variables are de Bruijn *levels* (the convention used in `Contender.v`): index 0 denotes the *outermost* binder.

7.2 Depth measure

Term depth is exactly the same recursive measure as in `Contender.v`, with one new leaf:

```

Fixpoint term_depth (t : term) : nat :=
  match t with
  | tVar x ⇒ S (Contender.nat_depth x)
  | tLam A B body ⇒
      S (max (max (Contender.type_depth A) (Contender.type_depth B))
          (term_depth body))
  | tApp t1 t2 ⇒ S (max (term_depth t1) (term_depth t2))
  | t0 ⇒ 1
  | tS ⇒ 1
  | tNatRec R ⇒ S (Contender.type_depth R)
  | tGrow ⇒ 1
  end.

```

`tGrow` has depth 1, just like `t0` and `tS`: it is a constant. This is what lets the witness term `tGrow (tS (embed  $t^*$ ))` stay within depth 44 when $\text{depth}(t^*) \leq 42$ (one `tS` application, one `tGrow` application, each adding 1).

7.3 Denotational semantics

Types are interpreted as Coq types:

$$\llbracket \text{tpNat} \rrbracket = \mathbb{N}, \quad \llbracket \text{tpArr } A \ B \rrbracket = \llbracket A \rrbracket \rightarrow \llbracket B \rrbracket.$$

A *pack* is a dependent pair $\{A : \text{type} \ \& \ [A]\}$ – a type tag together with a value at that type. Environments are lists of packs; the look-up of a de Bruijn level returns either `Some pack` or `None`.

The interpretation function recurses structurally on the term; we use the local aliases `cast B` (a partial coercion from `interp_type` from to `interp_type` to, returning `error` on type mismatch) and `errortp` (a default value at every type, 0 at `tpNat`, the constant function returning `error` at arrow types). Both come from `Contender.v`; we re-use them verbatim.

```
Fixpoint interp_term (e : list pack) (t : term) : pack :=
  match t with
  | tVar x =>
    match Contender.lookup e x with
    | Some R => R
    | None => existT _ tpNat error
    end
  | tLam A B body =>
    existT _ (tpArr A B)
    (fun x' => cast B (projT2 (interp_term ((existT _ A x') :: e) body)))
  | tApp t1 t2 =>
    let '(existT _ R1 r1) := interp_term e t1 in
    let '(existT _ R2 r2) := interp_term e t2 in
    match R1 as R1' return interp_type R1' -> pack with
    | tpNat => fun _ => existT _ tpNat error
    | tpArr A B => fun r1 => existT _ B (r1 (cast A r2))
    end r1
  | t0 => existT _ tpNat 0
  | tS => existT _ (tpArr tpNat tpNat) S
  | tNatRec R =>
    existT _ (tpArr R (tpArr (tpArr tpNat (tpArr R R)) (tpArr tpNat R)))
    (@Nat.recursion (interp_type R))
  | tGrow => existT _ (tpArr tpNat tpNat) G.grow
  end.
```

The five System T cases are an almost line-for-line port of `Contender.interp_term`. The new case, `tGrow`, returns the engine `G.grow` – which we will instantiate with `BigGrow` at the top level.

The top-level *evaluator* extracts a natural number, returning 0 when the closed term has an arrow type:

```
Definition eval (t : term) : nat :=
  let (tp, res) := interp_term nil t in
  match tp as t0 return (interp_type t0 -> nat) with
  | tpNat => fun res : interp_type tpNat => res
  | tpArr _ _ => fun _ => 0
  end res.
```

7.4 Depth-bounded enumeration and the contender

The enumerator `termsUpTo` lists every closed term of depth $\leq n$, segment by segment:

```

Fixpoint termsUpTo (n : nat) : list term :=
  match n with
  | 0 => []
  | S m =>
    List.map tVar (Contender.natsUpTo m) ++
    List.map (fun '(A, B, body) => tLam A B body)
      (list_prod (list_prod (Contender.typesUpTo m) (Contender.typesUpTo m))
        (termsUpTo m)) ++
    List.map (fun '(t1, t2) => tApp t1 t2)
      (list_prod (termsUpTo m) (termsUpTo m)) ++
    [t0] ++ [tS] ++
    List.map tNatRec (Contender.typesUpTo m) ++
    [tGrow]
end.

```

Completeness of the enumerator (*every* depth-bounded term is listed) is proved as `termsUpTo_complete`. We reuse `Contender.maxBy` (a polymorphic max-by-key combinator) to define the largest evaluator, and finally:

```

Definition largest_of_depth (n : nat) : term :=
  Contender.maxBy eval 0 t0 (termsUpTo n).

Definition largest_Grow_nat_of_depth (n : nat) : nat :=
  eval (largest_of_depth n).

(* ... wrapped in Module GrowEmbed (G : GrowSig), then applied with: *)
Definition contender_6 : nat := BigGrowEmbed.contender_grow_6.

```

The unfolded form of `contender_6`, after delta-reducing the helpers, is

$$\text{contender_6} = \text{eval}_{\mathcal{L}_{\text{Grow}}}(\text{maxBy eval}_{\mathcal{L}_{\text{Grow}}} 0 t0 (\text{termsUpTo } 44)).$$

Cross-checking with `cbv beta delta [...] in contender_6` confirms that *none* of `contender_5`, `largest_STLCNatRec_nat_of_depth`, `Contender.term`, or `Contender.tNatRec` appears in the unfolded form. The definition is, in the technical sense intended by the contest, *clean*.

8 The proof: a high-level map

The strict inequality `contender_5 < contender_6` factors through one explicit lower bound:

$$\text{contender_5} < \text{BigGrow}(\text{contender_5} + 1) \leq \text{contender_6}.$$

The first inequality is one line, by `BigGrow_gt_S`. The second is the heart of the proof; it splits into three sub-claims, each treated in its own section below.

(A) Existential maximizer. There is a closed System T term t^* with $\text{depth}(t^*) \leq 42$ and $\text{eval}(t^*) = \text{contender_5}$. This is the lemma `exists_maximizer_42`.

(B) Embedding preserves evaluation. There is an embedding `embed : Contender.term → term` such that $\text{eval}(\text{embed } t) = \text{Contender.eval}(t)$ for every closed term t . This is the lemma `embed_eval`, proved via the logical relation.

(C) **Witness construction.** Set

$$w(t^*) := \text{tApp tGrow (tApp tS (embed } t^*)).$$

Two further lemmas pin down the size and value of this witness: `witness_grow_depth` establishes $\text{depth}(w(t^*)) \leq 44$, and `witness_grow_eval` establishes $\text{eval}(w(t^*)) = \text{BigGrow}(\text{contender_5} + 1)$.

Combining (A), (B), (C) with `lowerbound_maxBy` gives the explicit bound $\text{BigGrow}(\text{contender_5} + 1) \leq \text{contender_6}$, hence the strict inequality.

We treat each of (A), (B), (C) in turn.

9 (A) The existential maximizer

The first ingredient is the inhabited witness theorem:

```
Lemma exists_maximizer_42 :
  exists tstar : Contender.term,
    Contender.term_depth tstar ≤ 42 ∧
    Contender.eval tstar = Contender.contender_5.
```

The proof is short, but it leans on a small auxiliary lemma:

```
Lemma contender_5_ge_1 : 1 ≤ Contender.contender_5.
Proof.
  unfold Contender.contender_5, Contender.largest_STLCNatRec_nat_of_depth,
    Contender.largest_of_depth.
  change 1 with (Contender.eval (Contender.tApp Contender.tS Contender.t0)).
  apply Contender.lowerbound_maxBy
    with (x := Contender.tApp Contender.tS Contender.t0);
    [apply (proj1 (Contender.termsUpTo_correct 42 _)); cbv; lia | reflexivity].
Qed.
```

The witness `tApp tS t0` has depth 3 (well within 42), evaluates to 1, and so by `lowerbound_maxBy` we get $1 \leq \text{contender_5}$.

With this in hand, `exists_maximizer_42` is a case analysis on `Contender.maxBy_In`:

```
Lemma exists_maximizer_42 :
  exists tstar : Contender.term,
    Contender.term_depth tstar ≤ 42 ∧
    Contender.eval tstar = Contender.contender_5.
Proof.
  pose proof contender_5_ge_1 as Hge.
  unfold Contender.contender_5, Contender.largest_STLCNatRec_nat_of_depth,
    Contender.largest_of_depth in *.
  exists (Contender.maxBy Contender.eval 0 Contender.t0
    (Contender.termsUpTo 42));
    split; [|reflexivity].
  destruct (Contender.maxBy_In Contender.eval (Contender.termsUpTo 42)
    0 Contender.t0 eq_refl) as [Pin | Peq].
  - apply (proj2 (Contender.termsUpTo_correct 42 _)) in Pin. exact Pin.
  - rewrite Peq in Hge. cbv in Hge. lia.
Qed.
```

There are two cases: either the `maxBy` returned an element of `termsUpTo 42` (whose depth is bounded by 42 via `termsUpTo_correct`), or it returned the initial best `t0`. The latter case forces `contender_5 = eval t0 = 0`, contradicting `contender_5_ge_1`.

Remark 9.1. The lemma is purely an *existence* statement – there is no algorithm here that would extract t^* in any reasonable amount of time. Computing the actual optimal t^* requires running the depth-42 enumeration, which is astronomical. The proof only needs the abstract existence, and the proof script is therefore short.

10 (B) Embedding preserves evaluation: the logical relation

We need

$$\forall t \in \text{Contender.term}, \quad \text{eval}_{\mathcal{L}_{\text{Grow}}}(\text{embed}(t)) = \text{Contender.eval}(t).$$

The embedding itself is the obvious congruence map (every constructor of `Contender.term` maps to the corresponding constructor of $\mathcal{L}_{\text{Grow}}$):

```
Fixpoint embed_term (t : Contender.term) : term :=
  match t with
  | Contender.tVar x => tVar x
  | Contender.tLam A B body => tLam A B (embed_term body)
  | Contender.tApp t1 t2 => tApp (embed_term t1) (embed_term t2)
  | Contender.t0 => t0
  | Contender.tS => tS
  | Contender.tNatRec R => tNatRec R
end.
```

Depth is obviously preserved:

```
Lemma term_depth_embed : forall t,
  term_depth (embed_term t) = Contender.term_depth t.
Proof. induction t; simpl; congruence. Qed.
```

10.1 Why a logical relation?

The naive equational claim

$$\text{Contender.interp_term } e_1 t = \text{interp_term } e_2 (\text{embed } t)$$

fails as stated: the two interpretations have type $\{tp : \text{type} \& [tp]\}$, and at function types two extensionally equal λ -expressions are not *intensionally* equal in Coq without function extensionality. We do not want to introduce `FunctionalExtensionality` (it is an axiom; the contest forbids those), so we replace propositional equality of *values* by a Tait-style logical relation that collapses to plain equality at `tpNat`:

```
Fixpoint RelVal (A : type) : interp_type A → interp_type A → Prop :=
  match A with
  | tpNat => fun x y => x = y
  | tpArr A1 A2 => fun f g => forall x y, RelVal A1 x y → RelVal A2 (f x) (g y)
end.
```

At `tpNat`, $\text{RelVal } x y \iff x = y$. At `tpArr A B`, two functions are related iff they map related arguments to related results. This is exactly Tait's *computability predicate*, which gives strong normalization for System T; we will use it as an extensional substitute for equality.

The relation lifts to packs (a pack-level relation that asserts *both packs have the same type tag* and the values are related at that tag) and to environments (pointwise via `Forall12`):

```

Definition RelPack (p q : pack) : Prop :=
  exists tp (v1 v2 : interp_type tp),
    p = existT Contender.interp_type tp v1
    ∧ q = existT Contender.interp_type tp v2
    ∧ RelVal tp v1 v2.

Definition RelEnv (e1 e2 : list pack) : Prop :=
  List.Forall12 RelPack e1 e2.

```

A small one-line constructor `RelPack_intro` lets us build a `RelPack` from a pair of packs at the same tag:

```

Lemma RelPack_intro :
  forall tp (v1 v2 : interp_type tp),
    RelVal tp v1 v2 →
    RelPack (existT Contender.interp_type tp v1)
             (existT Contender.interp_type tp v2).

```

10.2 Two technical facts

Two routine lemmas about `RelVal` feed into the main induction:

```

Lemma error_related : forall tp, RelVal tp (@error tp) (@error tp).
Proof. induction tp; simpl; [reflexivity | intros _ _ _; exact IHtp2]. Qed.

```

The error value is related to itself at every type. At `tpNat` this is just $0 = 0$; at arrow types we use the IH on the codomain.

The second fact concerns `cast`: it preserves the relation when applied to related arguments. The proof is by induction on the `from/to` pair; the only non-trivial case is `tpArr → tpArr`, where the cast wraps its argument backwards (covariance/contravariance), so we have to prove *both* directions of the cast simultaneously:

```

Lemma cast_impl_related : forall from to,
  (forall v1 v2, RelVal from v1 v2 →
    RelVal to (fst (Contender.cast_impl from to) v1)
              (fst (Contender.cast_impl from to) v2))
  ∧
  (forall u1 u2, RelVal to u1 u2 →
    RelVal from (snd (Contender.cast_impl from to) u1)
                (snd (Contender.cast_impl from to) u2)).

Lemma cast_related : forall from to v1 v2,
  RelVal from v1 v2 →
  RelVal to (@cast from to v1) (@cast from to v2).
Proof. intros; apply (proj1 (cast_impl_related _ _)); assumption. Qed.

```

10.3 Look-up under related environments

The look-up function `Contender.lookup e n` has a clean specification under related environments: *either both look-ups miss, or both succeed and return related packs*.

```

Lemma lookup_related :
  forall e1 e2 n,

```

```

RelEnv e1 e2 →
match Contender.lookup e1 n, Contender.lookup e2 n with
| Some p1, Some p2 ⇒ RelPack p1 p2
| None, None ⇒ True
| _, _ ⇒ False
end.

```

The proof is a few lines: `Forall2` forces $|e_1| = |e_2|$, so the bounds-check on `lookup` agrees on both sides, and an auxiliary `Forall2_nth_error` lemma transports the related-pack assertion through `nth_error`.

10.4 The main induction

The fundamental theorem of the embedding is stated symmetrically as “*related environments map related terms*”:

```

Lemma embed_interp_related :
  forall e1 e2 t,
    RelEnv e1 e2 →
    RelPack (Contender.interp_term e1 t)
             (interp_term e2 (embed_term t)).

```

The proof inducts on $t : \text{Contender.term}$. We sketch each case.

Case `tVar`. Both sides reduce to a `match` on `Contender.lookup  $e_i$   $x$` ; `lookup_related` gives exactly the four cases we need (Some/Some, Some/None, None/Some, None/None), and the two one-sided cases are killed by `contradiction`. In Some/Some we use the supplied `RelPack`; in None/None we use `RelPack_error` (a trivial corollary of `error_related`).

Case `tLam`. Both sides reduce to `existT (tpArr  $A$   $B$ ) ( $\lambda x'. \dots$ )`; we apply `RelPack_intro` at the tag `tpArr  $A$   $B$` and then prove the function bodies are pointwise related. The IH gives us that, given $\text{Henv}' : \text{RelEnv} (p_1 :: e_1) (p_2 :: e_2)$ for a related pair (p_1, p_2) extending the original environments, the bodies evaluate to a `RelPack`; we then push that through `cast_related`.

Case `tApp`. Both sides reduce to a `match` on the argument-1 type tag; we use the two IHs to obtain the function and the argument as a pair of `RelPacks`, destruct the type tag, and discharge the `tpArr` case via the function-relation property of `RelVal` at arrow types. The non-arrow branches fall to `RelPack_error`.

Case `t0`. `RelPack_intro` at `tpNat` with witness $0 = 0$.

Case `tS`. `RelPack_intro` at `tpArr tpNat tpNat` with the function relation $\forall x y, x = y \Rightarrow Sx = Sy$, which is just `f_equal`.

Case `tNatRec`. This is the most interesting case: we need that `Nat.recursion` maps related triples (base, step, counter) to related accumulators. A standard nested induction on the natural-number counter – the base case uses the related base values, the successor case uses the step relation – does the job.

10.5 From the relation to actual equality

Because `RelVal tpNat  $x$   $y$` is just $x = y$, the fundamental theorem specialized to closed terms gives us *exact* equality of the natural number outputs:

```

Lemma embed_eval : forall t,
  eval (embed_term t) = Contender.eval t.
Proof.
  intro t.
  pose proof (embed_interp_related [] [] t (List.Forall2_nil _))
    as (tp & v1 & v2 & Hp1 & Hp2 & Hrel).
  unfold eval, Contender.eval.
  replace (Contender.interp_term nil t) with (existT Contender.interp_type tp v1)
    by (symmetry; exact Hp1).
  replace (interp_term nil (embed_term t)) with (existT Contender.interp_type tp v2
    )
    by (symmetry; exact Hp2).
  destruct tp; [symmetry; exact Hrel | reflexivity].
Qed.

```

The empty environment is trivially related to itself (`Forall2_nil`); the fundamental theorem then yields a common type tag tp and a pair of values (v_1, v_2) with `RelVal tp v1 v2`. At the only tag where the top-level evaluator extracts a number – `tpNat` – `RelVal` is plain equality, hence $v_1 = v_2$, hence the two evaluators agree. At any other tag both sides return 0.

10.6 What we did *not* need

It is worth highlighting what this proof did *not* need: *function extensionality*. At no point did we conclude $\lambda x. F(x) = \lambda x. G(x)$ from $\forall x. F(x) = G(x)$. Instead, the logical relation kept us at the relation level until we projected at `tpNat`. That is the crux of the no-axioms invariant.

11 (C) The witness term

11.1 Construction

Given a System T term t^* – the existential maximizer of (A) – the witness is

$$w(t^*) := \text{tGrow}(\text{tS}(\text{embed } t^*))$$

i.e. the depth-3 frame `tApp tGrow (tApp tS •)` applied to `embed t*`.

```

Definition witness_grow (tstar : Contender.term) : term :=
  tApp tGrow (tApp tS (embed_term tstar)).

```

11.2 Depth is at most 44

```

Lemma witness_grow_depth :
  forall tstar,
    Contender.term_depth tstar ≤ 42 →
    term_depth (witness_grow tstar) ≤ 44.
Proof.
  intros tstar H. unfold witness_grow. simpl. rewrite term_depth_embed.
  destruct (Contender.term_depth tstar); simpl; lia.
Qed.

```

The proof is by computation: `term_depth` on the outer `tApp tGrow` adds 1 to the maximum of `depth(tGrow) = 1` and `depth(tApp tS ...)`. The inner application adds 1 to `max(depth(tS) = 1, depth(embed t*))`. By `term_depth_embed`, this last quantity equals `depth(t*)`, which is

bounded by 42. Working out the arithmetic: the depth is $1 + \max(1, 1 + \max(1, \text{depth } t^*))$. A small case split on whether $\text{depth } t^* = 0$ (impossible because variables and constructors are at least 1) or ≥ 1 discharges the `Nat.max` simplifications, and `lia` finishes.

11.3 Eval of the witness is `BigGrow(contender_5 + 1)`

The semantic content of the witness is:

```

Lemma witness_grow_eval :
  forall tstar,
    Contender.eval tstar = Contender.contender_5 →
    eval (witness_grow tstar) = G.grow (S Contender.contender_5).
Proof.
  intros tstar Heq.
  assert (Hembed : eval (embed_term tstar) = Contender.contender_5)
    by (rewrite embed_eval; exact Heq).
  unfold witness_grow, eval in *.
  destruct (interp_term [] (embed_term tstar)) as [[|A B] res] eqn:E;
  simpl in Hembed.
  - subst res. cbn [interp_term]; rewrite E; cbn [interp_term]; reflexivity.
  - pose proof contender_5_ge_1; lia.
Qed.

```

The argument is an extended computation. We assume `Contender.eval t* = contender_5`. By `embed_eval`, `eval(embed t*) = contender_5` too, so the closed term `embed t*` interprets to a pack of the form `existTtpNat contender_5` – in the `tpNat` branch. Plug this into the chain of two `tApps`:

- Inner: `interp_term [] (tApp tS (embed t*)) = existTtpNat (S contender_5)`.
- Outer: `interp_term [] (tGrow ...) = existTtpNat (G.grow (S contender_5))`.

The final `eval` extracts the `tpNat` value, giving exactly `G.grow(S contender_5)`. Mechanically this reduces to two `cbn [interp_term]; rewrite E` steps.

The arrow case in the destruction is impossible: it would force `eval(embed t*) = 0`, but `eval(embed t*) = contender_5 ≥ 1`.

12 The main theorem

We now have all three ingredients. The first half of the chain is `grow_lower_bound`:

```

Theorem grow_lower_bound :
  G.grow (S Contender.contender_5) ≤ contender_grow_6.
Proof.
  unfold contender_grow_6, largest_Grow_nat_of_depth, largest_of_depth.
  destruct exists_maximizer_42 as (tstar & Hdepth & Heval).
  rewrite ← (witness_grow_eval tstar Heval).
  eapply Contender.lowerbound_maxBy with (x := witness_grow tstar). 2: reflexivity.
  apply (termsUpTo_complete 44 (witness_grow tstar)).
  apply witness_grow_depth. exact Hdepth.
Qed.

```

Reading the script: we want `G.grow(S contender_5) ≤ eval (maxBy eval 0 t0 (termsUpTo 44))`. We extract `t*` via `exists_maximizer_42` and rewrite the LHS as `eval(w(t*))` via `witness_grow_eval`. Then `lowerbound_maxBy` with witness `w(t*)` reduces the goal to (i) *w(t^{*}) is in termsUpTo 44*,

which follows from `termsUpTo_complete` and `witness_grow_depth`, and (ii) $\text{eval}(t0) = 0$, by reflexivity.

The strict inequality follows immediately:

```
Theorem contender_5_lt_contender_grow_6 :
  Contender.contender_5 < contender_grow_6.
Proof.
  eapply Nat.lt_le_trans with (m := G.grow (S Contender.contender_5)).
  - apply G.grow_gt_S.
  - exact grow_lower_bound.
Qed.
```

By `G.grow_gt_S`, `contender_5 < G.grow(S contender_5)`; combined with `grow_lower_bound` this gives `strict < contender_grow_6`.

12.1 Generic vs. instantiated

The theorem inside `Module GrowEmbed (G : GrowSig)` is parameterized over an arbitrary growth engine G satisfying $\forall n, n < G.\text{grow}(Sn)$. At the top level we instantiate G with `BigGrow`:

```
Module BigGrowSig <: GrowSig.
  Definition grow := BigGrow.
  Definition grow_gt_S : forall n, n < grow (S n) := BigGrow_gt_S.
End BigGrowSig.

Module BigGrowEmbed := GrowEmbed(BigGrowSig).

Definition contender_6 : nat := BigGrowEmbed.contender_grow_6.

Theorem contender_5_lt_contender_6 : contender_5 < contender_6.
Proof. exact BigGrowEmbed.contender_5_lt_contender_grow_6. Qed.
```

The functor structure is bookkeeping: the entire proof goes through abstractly over G , and the `BigGrow` specialization is two lines of glue.

13 Discussion

13.1 Cleanliness

The crucial fact for contest acceptance is that the unfolded `contender_6` does not mention `contender_5` in any form. A mechanical check via `cbv beta delta [...]` on the chain

`contender_6` $\rightsquigarrow$ `contender_grow_6` $\rightsquigarrow$ `largest_Grow_nat_of_depth44` $\rightsquigarrow$ `eval (Contender.maxBy eval (...))`

confirms that, after every legitimate δ -reduction, the result contains no `contender_5`, no `largest_STLCNatRec_nat_of_depth`, no `Contender.term` constructor, and no other artifact of the previous contender’s machinery. The polymorphic `maxBy` is shared because it is a generic combinator – it cares only about a key function and a list, not what those things are.

The *proof* of strict inequality, by contrast, freely uses the existential maximizer of `contender_5` (via `maxBy_In`), as well as `Contender.interp_term`, `eval`, `cast_impl`, and `lookup`. This is exactly the “definitional cleanliness, proof witness reuse” design the contest’s spirit was meant to encourage.

13.2 How big is contender_6?

The explicit lower bound we proved is

$$\text{contender_6} \geq \text{BigGrow}(\text{contender_5} + 1) = f_{\varepsilon_0}(\text{contender_5} + 1).$$

Now `contender_5` is itself a number that, while we cannot compute it explicitly within any reasonable time bound, is known to exceed $f_\alpha(42)$ for every $\alpha < \omega^\omega$ (and almost certainly more, since System T can encode much faster-growing functions). Plugging an Ackermann-scale number into a function on the f_{ε_0} scale gives a value that completely dwarfs any contender below us in the chain, and dwarfs them by a margin that grows faster than any *System T*-definable function in `contender_5`. This is the “decisive margin” the contest README hopes for.

13.3 What about contender_7?

The fresh-engine pattern can be iterated, but each iteration faces a gradually rising bar. The natural next step is to replace `BigGrow` by an even faster-growing engine: f_{ε_ω} , f_{ζ_0} , f_{Γ_0} , or higher Veblen ordinals. All of these can be presented in the Brouwer style in Coq, but the `Brouwer` type itself does not enforce well-foundedness extensionally, so writing down the right *fundamental sequence* becomes the active design question rather than the recursion structure.

A second axis is the *connector lemma*: an actual proof that every closed System T term of depth d is bounded by $f_\alpha(d)$ for some $\alpha < \varepsilon_0$. Such a lemma would let us drop the embedding altogether and write `contender_N` := `BigGrow N` for some N . This is the cleanest possible submission shape, but the proof is a significant exercise in proof-theoretic ordinal analysis and is well out of scope here.

13.4 Summary of dependencies

The whole submission is self-contained within `Contender.v` and introduces no external libraries beyond `Arith`, `Lia`, and `List` (already required by the file). No `FunctionalExtensionality`, no `ProofIrrelevance`, no `ClassicalLogic`. Both `Print Assumptions contender_4_lt_contender_5` and `Print Assumptions contender_5_lt_contender_6` report “*Closed under the global context*”.

The whole file type-checks in roughly 5 seconds on a modern laptop running Rocq 9.0.1, well within the contest’s 15 s/60 s budgets, and `coqchk Contender` accepts the resulting modules.

A Pointer to the Coq source

The full Coq source corresponding to this article is the file `Contender.v` on the `submit-c6-prep` branch, lines 830–1405 (the new contender 6 section, including the inlined Brouwer ordinals and the $\mathcal{L}_{\text{Grow}}$ functor). Each identifier mentioned in this article matches the corresponding Coq identifier verbatim.

References

- [1] Scott Aaronson. *Who Can Name the Bigger Number?* <https://www.scottaaronson.com/writings/bignumbers.html>, 1999.

- [2] The Coq Development Team. *The Coq Reference Manual*. <https://coq.inria.fr/>, version 8.20 / Rocq 9.
- [3] W. W. Tait. *Intensional Interpretations of Functionals of Finite Type I*. Journal of Symbolic Logic, 32(2): 198–212, 1967.
- [4] H. Schwichtenberg, S. Wainer. *Proofs and Computations*. Cambridge University Press, 2012. (See ch. 4 for Brouwer ordinals and the fast-growing hierarchy.)
- [5] Cody Roux. *name-the-biggest-number*. <https://github.com/codyroux/name-the-biggest-number>.